`hdc_core_top.sv` — End-to-End Inference Core and Co-Simulation Flow

This note documents the **complete inference core** of the 1024-HDC project: quantized EMG window in, gesture class label out. It is the capstone integration that wires the verified encoder (`encoder_top`) into the verified associative memory (`popcount_am`), and it is proven bit-for-bit against the Python golden reference running the *same end-to-end computation*.

---

1. What the core computes

With the item memories, trained class prototypes, and pruning mask in place, one inference is:

```
query      = encode_emg_window(levels)          # encoder_top
idx, dist  = classify(query, protos, mask)      # popcount_am
```

i.e. exactly `HDCEngine.encode_emg_window` followed by `HDCEngine.classify` from `hdc_ref.py` . This is the full datapath that will eventually run on the Zynq PL: the PS streams quantized feature windows in and reads predicted labels back.

---

2. The RTL: `rtl/hdc_core_top.sv`

`hdc_core_top` is a thin, zero-logic composition of the two verified blocks — deliberately so. All the functional complexity already passed its own co-sim; this module only adds the wiring and the configuration passthrough.

Interface

Signal	Dir	Width	Meaning
`clk` , `rst_n`	in	1	clock / async reset
`proto_we` , `proto_idx` , `proto_vec`	in	1 / 3 / 1024	prototype write port (one per clock)
`mask_we` , `mask_vec`	in	1 / 1024	pruning-mask write port (defaults to all-ones)
`start` , `levels_flat`	in	1 / 80	begin inference on a packed 4x5 level grid
`busy`	out	1	encode in progress
`out_valid`	out	1	pulses with the decision
`class_idx`	out	3	predicted class (nearest prototype)
`class_dist`	out	11	its masked Hamming distance

Composition

```
                +-----------------------------------------+
levels_flat -->| encoder_top                             |
  start     -->|  item_mem x3 -> bind ^ permute -> bundle    |
               |        (20 pairs, FSM)                  |
               +--------------------+--------------------+
                                    | query_vec / out_valid
                                    v
               +-----------------------------------------+
proto_we   --->| popcount_am                             |
mask_we    --->|  masked Hamming x N_CLASS -> argmin         |
               +--------------------+--------------------+
                                    v
                          out_valid / class_idx / class_dist
```

The encoder's `out_valid` / `query_vec` drive the AM's `q_valid` / `query_vec` directly — no buffering needed, since the AM registers its decision in one cycle. Total latency: encode (N_PAIRS + ~3 control cycles) + 1 cycle classify ≈ **24 clocks per inference** at the default 4x5 configuration.

---

3. Verification — "Python is truth", now end-to-end

The earlier co-sims proved each block in isolation. This one proves the *composition*: any mismatch in hand-off timing, query bit-ordering, or configuration plumbing between encoder and AM would surface here.

Golden vectors — `python_ref/generate_vectors.py --core`

`hdc_ref.export_core_cosim` mirrors a real deployment:

1. Builds the deterministic `ItemMemory` and writes the three `.mem` files (the DUT's ROMs initialise from the *same* files).
2. **Trains** the 8 class prototypes — each is the majority bundle of 5 encoded random windows — rather than using arbitrary random prototypes, so the distances exercised are the realistic "query is similar to several prototypes" regime, not the trivially-orthogonal one.
3. Draws one pruning mask (~75 % keep density).
4. For each of `count` cases: encodes a fresh random window and classifies it, recording `(best_idx << 16) | best_dist`.

File	Contents
`item_mem_*.mem`	channel / feature / value tables (DUT ROM init)
`core_proto.hex`	8 trained prototypes, class-major
`core_mask.hex`	the pruning mask
`core_levels.hex`	packed level grid per case
`core_expect.hex`	`(best_idx << 16) | best_dist` per case

Testbench — `tb/tb_core_cosim.sv`

Mirrors the deployment protocol: after reset it loads the 8 prototypes and the mask **once** (`configure_core`), then loops the cases — drive `levels_flat` , pulse `start` , wait for `out_valid` , compare **both** `class_idx` and `class_dist` . Mismatch ⇒ `$fatal` (non-zero exit); clean sweep ⇒ `$finish` .

Harness — `sim/run_core_cosim.do`

```
vsim -c -do sim/run_core_cosim.do      # NUM_CASES env var overrides count (default
500)
```

One command: regenerate golden → fresh `work` → compile the five RTL files (`item_mem` , `bundle_unit` , `encoder_top` , `popcount_am` , `hdc_core_top`) + TB → simulate.

---

4. Result

```
tb_core_cosim: end-to-end inference vs Python golden
  VECDIR = python_ref/vectors/cosim_core
  CASES  = 500   (D=1024, 4x5 pairs, N_CLASS=8)
PASS: all 500 end-to-end inference cases match the Python golden.
```

500 / 500 inferences — trained prototypes, ~75 % pruning mask, full encode → classify path — match the Python golden exactly on ModelSim SE-64 10.6e.

5. Where this leaves the project

The complete HDC inference datapath is now verified in RTL:

Block	Co-sim	Status
`xor_permute_top` (bind + permute)	`run_cosim.do`	PASS
`bundle_unit` (majority bundle)	`run_bundle_cosim.do`	PASS
`popcount_am` (associative memory)	`run_am_cosim.do`	PASS
`encoder_top` + `item_mem` (window encoder)	`run_encoder_cosim.do`	PASS
`hdc_core_top` (end-to-end inference)	`run_core_cosim.do`	**PASS**

What remains is **interface and platform work**, not algorithm work: an AXI wrapper exposing the core's configuration and inference ports to the Zynq PS (AXI4-Lite for control, AXI4-Stream + DMA for streaming windows), the bare-metal driver, and the on-board measurements (throughput / latency / energy / area) that feed the novelty studies.